

Das 12-Münzen-Problem in Java: Simulation, Strategien und objektorientiertes Design

Benjamin Malicke ©

7. März 2026

Inhaltsverzeichnis

1	Einleitung	3
2	Grundstruktur der Simulation	3
2.1	Strategie-Schnittstelle <code>CoinWeighingStrategy</code>	3
2.2	Simulationsklasse mit Strategie-Auswahl	4
3	Die feste Entscheidungsstrategie <code>FixedTreeStrategy</code>	5
3.1	Hilfsmethode zum Wiegen	5
3.2	Überblick über den Entscheidungsbaum	6
4	Vergleichsstrategie <code>RandomGuessStrategy</code>	7
4.1	Idee der Zufallsstrategie	7
4.2	Implementierung von <code>RandomGuessStrategy</code>	8
4.3	Einsatz in der Simulation	8
5	Tests der fest verdrahteten Strategie	9
5.1	Einfache Testklasse ohne JUnit	9
5.2	Hinweis auf JUnit-Tests	10
6	Verbesserung der Lesbarkeit mit <code>enum WeighResult</code>	11
6.1	Definition des Enums <code>WeighResult</code>	11
6.2	Angepasste Hilfsmethode <code>weigh</code>	12
6.3	Verwendung des Enums in der Strategielogik	12
6.4	Vorteile des Enum-Ansatzes	13
7	Zusammenfassung und Ausblick	13
A	Vollständiger Quelltext	14
A.1	Interface <code>CoinWeighingStrategy</code>	15
A.2	Enum <code>WeighResult</code> (optionale Verbesserung)	15
A.3	Simulationsklasse <code>CoinWeighingProblemWithStrategy</code>	15
A.4	Strategieklassse <code>FixedTreeStrategy</code>	16
A.5	Alternative Implementierung mit <code>WeighResult</code>	22
A.6	Vergleichsstrategie <code>RandomGuessStrategy</code>	28
A.7	Testklasse <code>FixedTreeStrategyTest</code>	28
A.8	Alte Implementierung ohne Strategy-Pattern	29
A.9	Entscheidungsbaum zur Lösungsstrategie	33
A.10	Entscheidungsbaum für Fall A (Falschmünze in 9–12)	33
A.11	Entscheidungsbaum für Fall B (1–4 schwerer als 5–8)	34

A.12 Entscheidungsbaum für Fall C (5–8 schwerer als 1–4)	34
--	----

1 Einleitung

In diesem Dokument wird das klassische *12-Münzen-Problem* untersucht und mit Hilfe von Java-Programmen simuliert. Ziel ist es, eine gefälschte Münze unter zwölf Münzen zu identifizieren. Die Falschmünze kann entweder schwerer oder leichter als die echten Münzen sein. Es stehen genau drei Wägungen auf einer Balkenwaage zur Verfügung.

Neben der mathematischen Struktur des Problems liegt der Schwerpunkt auf folgendem:

- Entwicklung einer Simulationsumgebung in Java,
- Vergleich verschiedener Strategien zur Identifikation der Falschmünze,
- Einsatz von **Interfaces** (z.B. `CoinWeighingStrategy`) zur sauberen Trennung von Simulation und Strategie,
- schrittweiser Übergang von „Skript-Code“ zu objektorientiertem Design.

Zur Auswertung der Strategien werden über viele zufällige Durchläufe („Simulationen“) Statistiken erhoben, insbesondere die Trefferquote (Genauigkeit) einer Strategie.

Im Folgenden werden zunächst die Grundidee der Simulation und die Verwendung einer Strategie-Schnittstelle erläutert. Danach werden konkrete Implementierungen wie `FixedTreeStrategy` und `RandomGuessStrategy` vorgestellt und getestet.

2 Grundstruktur der Simulation

Das Java-Programm besteht aus zwei klar getrennten Teilen:

1. einer *Simulationsumgebung*, die wiederholt zufällige Konstellationen des 12-Münzen-Problems erzeugt und die Trefferquote einer Strategie misst,
2. einer oder mehreren *Strategien*, die versuchen, die Falschmünze mit höchstens drei Wägungen zu identifizieren.

Um diese Trennung sauber auszudrücken, wird ein Interface `CoinWeighingStrategy` eingeführt. Die Simulationsklasse kennt nur dieses Interface und nicht die konkrete Implementierung. So können verschiedene Strategien (z.B. eine feste Entscheidungslogik oder ein reines Zufallsraten) leicht ausgetauscht und verglichen werden.

2.1 Strategie-Schnittstelle `CoinWeighingStrategy`

Das Interface definiert nur eine einzige Methode:

Listing 1: Strategie-Interface `CoinWeighingStrategy`

```
1  public interface CoinWeighingStrategy {
2      /**
3       * Versucht, die gefälschte Münze zu finden.
4       *
5       * @param coins Array mit Münzgewichten
6       * @param fakeIndex Index der tatsächlichen Falschmünze (für die
7       *   Auswertung)
8       * @param writer für Protokollausgaben (kann auch null sein)
9       * @return true, wenn Münze und "schwerer/leichter" korrekt
10      *   identifiziert wurden
11      */
12      boolean findFakeCoin(int[] coins, int fakeIndex,
13                          java.io.PrintWriter writer);
14  }
```

Die Idee ist:

- Das Array `coins` enthält die Gewichte aller Münzen. Echte Münzen haben z.B. Gewicht 10, die Falschmünze hat Gewicht 8 (leichter) oder 12 (schwerer).
- `fakeIndex` gibt an, an welcher Position sich die Falschmünze befindet (Index 0 bis 11).
- Über den optionalen `PrintWriter` können Strategien ihre Wägungen und Zwischenschritte protokollieren. In reinen Tests kann hier auch `null` übergeben werden.
- Der Rückgabewert ist `true`, wenn Strategie und Simulation übereinstimmen (d.h. richtiger Index und richtige Richtung: schwerer oder leichter), sonst `false`.

Damit ist die Schnittstelle bewusst allgemein gehalten: Sie schreibt nicht vor, *wie* eine Strategie die Falschmünze finden soll, nur, *dass* sie es versucht und am Ende mitteilt, ob sie korrekt war.

2.2 Simulationsklasse mit Strategie-Auswahl

Die Simulationsklasse erzeugt in einer Schleife immer wieder neue Zufallsfälle und übergibt diese an eine gewählte Strategie. Die Strategie wird über das Interface typisiert und kann dadurch leicht gewechselt werden.

Listing 2: Grundstruktur der Simulationsklasse

```
1  import java.io.FileWriter;
2  import java.io.IOException;
3  import java.io.PrintWriter;
4  import java.util.Random;
5
6  public class CoinWeighingProblemWithStrategy {
7
8      public static void main(String[] args) {
9          try (PrintWriter writer = new PrintWriter(new
10              FileWriter("output.txt"))) {
11              writer.println("Ziel ist es, eine gefälschte Münze zu
12                  finden.");
13              writer.println("Eine von 12 Münzen ist entweder zu leicht
14                  oder zu schwer. " +
15                  "Man darf nur dreimal wiegen.\n");
16
17              int simulationen = 1200;
18              int richtigeIdentifikationen = 0;
19
20              Random rand = new Random();
21              CoinWeighingStrategy strategy = new FixedTreeStrategy();
22
23              for (int sim = 0; sim < simulationen; sim++) {
24                  int[] coins = new int[12];
25                  for (int i = 0; i < 12; i++) {
26                      coins[i] = 10; // echte Münzen
27                  }
28
29                  int fakeCoin = rand.nextInt(12); // Index 0..11
30                  boolean isHeavier = rand.nextBoolean();
31                  coins[fakeCoin] = isHeavier ? 12 : 8; // Falschmünze setzen
32
33                  writer.println("\nSimulation gestartet. Eine Münze ist
34                      gefälscht.");
35              }
36          }
37      }
38  }
```

```

31         writer.println("\t(Für Testzwecke: Münze " + (fakeCoin + 1)
32             + " ist " +
33             (isHeavier ? "schwerer (12g)" : "leichter (8g)") + ")");
34         boolean korrekt = strategy.findFakeCoin(coins, fakeCoin,
35             writer);
36         if (korrekt) {
37             richtigeIdentifikationen++;
38         }
39     }
40     writer.println();
41     writer.println("Korrekte Identifikationen: " +
42         richtigeIdentifikationen);
43     writer.println("Genauigkeit: " +
44         (100.0 * richtigeIdentifikationen / simulationen) + "%");
45 } catch (IOException e) {
46     e.printStackTrace();
47 }
48 }

```

Wichtige Punkte dieser Struktur:

- Die Simulationsklasse kennt nur den Typ `CoinWeighingStrategy`, nicht die Details von `FixedTreeStrategy`.
- Durch Ersetzen von

```
1 CoinWeighingStrategy strategy = new FixedTreeStrategy();
```

z.B. durch

```
1 CoinWeighingStrategy strategy = new RandomGuessStrategy();
```

kann eine andere Strategie getestet werden, ohne die Simulationslogik zu verändern.

- Die Statistik (Anzahl korrekter Identifikationen und prozentuale Genauigkeit) ist vollständig von der eigentlichen Suchstrategie entkoppelt.

In den folgenden Abschnitten werden konkrete Implementierungen des Interfaces vorgestellt: eine fest verdrahtete Entscheidungsstrategie (`FixedTreeStrategy`) und eine einfache Zufallsstrategie (`RandomGuessStrategy`).

3 Die feste Entscheidungsstrategie `FixedTreeStrategy`

In diesem Abschnitt wird eine konkrete Strategie implementiert, die versucht, das 12-Münzen-Problem mit genau drei Wägungen zu lösen. Die Klasse `FixedTreeStrategy` implementiert dabei das bereits vorgestellte Interface `CoinWeighingStrategy` und enthält eine fest verdrahtete Entscheidungslogik (einen Suchbaum).

3.1 Hilfsmethode zum Wiegen

Zunächst wird eine Hilfsmethode `weigh` definiert, die zwei Münzgruppen auf der Balkenwaage vergleicht. Zurückgegeben wird, ob die linke Seite schwerer, leichter oder gleich schwer ist.

Listing 3: Hilfsmethode weigh in FixedTreeStrategy

```

1  import java.io.PrintWriter;
2
3  public class FixedTreeStrategy implements CoinWeighingStrategy {
4
5      /**
6       * Vergleicht die Summe zweier Münzgruppen.
7       *
8       * @param coins Array mit allen Münzgewichten
9       * @param left Indizes der Münzen auf der linken Seite der Waage
10      * @param right Indizes der Münzen auf der rechten Seite der Waage
11      * @return -1, wenn links leichter, 0 bei Gleichgewicht,
12      * 1, wenn links schwerer ist
13      */
14     private int weigh(int[] coins, int[] left, int[] right) {
15         int leftSum = 0, rightSum = 0;
16         for (int i : left) {
17             leftSum += coins[i];
18         }
19         for (int i : right) {
20             rightSum += coins[i];
21         }
22         return Integer.compare(leftSum, rightSum); // -1, 0, 1
23     }
24
25     //... (Strategielogik folgt unten)
26 }

```

Diese Methode abstrahiert das eigentliche Wiegen: Die Strategie muss nur noch angeben, welche Münzen auf die linke bzw. rechte Seite gelegt werden, nicht aber, wie das Ergebnis numerisch berechnet wird.

3.2 Überblick über den Entscheidungsbaum

Die eigentliche Strategiemethode `findFakeCoin` führt bis zu drei Wägungen durch. Je nach Ergebnis verzweigt der Code in unterschiedliche Fälle. Vereinfacht läuft die Logik wie folgt ab:

1. **Erste Wägung:** Münzen 1–4 gegen Münzen 5–8.
 - *Fall A:* Die Waage bleibt im Gleichgewicht. Dann muss die Falschmünze unter Münzen 9–12 sein.
 - *Fall B:* Die linke Seite ist schwerer. Dann ist entweder eine Münze aus 1–4 zu schwer oder eine Münze aus 5–8 zu leicht.
 - *Fall C:* Die rechte Seite ist schwerer. Symmetrisch zu Fall B: entweder 5–8 zu schwer oder 1–4 zu leicht.
2. **Zweite Wägung:** Je nach Fall A, B oder C werden unterschiedliche Münzkombinationen verglichen, um die Menge der Kandidaten weiter einzugrenzen.
3. **Dritte Wägung:** Die letzte Wägung unterscheidet zwischen den verbleibenden ein bis drei Kandidaten und bestimmt schließlich, welche Münze gefälscht ist und ob sie schwerer oder leichter ist.

Die Implementierung von `findFakeCoin` ist relativ lang, da alle möglichen Kombinationen der drei Wägungen explizit behandelt werden. Der folgende Code zeigt die komplette Methode in einer Version, die direkt mit der Simulationsklasse aus Abschnitt 2 zusammenarbeitet:

Listing 4: Strategiemethode findFakeCoin (Ausschnitt)

```

1  @Override
2  public boolean findFakeCoin(int[] coins, int fakeCoin, PrintWriter
   writer) {
3      int identifiedCoin = -1; // Index der identifizierten Münze
4      boolean identifiedHeavier = false; // ist sie schwerer?
5
6      // 1. Wägung: Münzen 1-4 vs 5-8
7      int[] left1 = {0, 1, 2, 3};
8      int[] right1 = {4, 5, 6, 7};
9      int result1 = weigh(coins, left1, right1);
10
11     if (writer != null) {
12         writer.println("Wägung 1: Münzen 1-4 vs. 5-8: " +
13             (result1 == 0 ? "gleich"
14             : (result1 == 1 ? "1-4 schwerer" : "5-8 schwerer")));
15     }
16
17     if (result1 == 0) {
18         // Fall A: 1-8 sind normal, Falschmünze ist in 9-12
19         //... (weitere Wägungen und Fallunterscheidungen)
20     } else if (result1 == 1) {
21         // Fall B: 1-4 schwerer als 5-8
22         //... (weitere Wägungen und Fallunterscheidungen)
23     } else {
24         // Fall C: 5-8 schwerer als 1-4
25         //... (weitere Wägungen und Fallunterscheidungen)
26     }
27
28     boolean isHeavier = coins[fakeCoin] == 12;
29     return identifiedCoin == fakeCoin && identifiedHeavier ==
        isHeavier;
30 }

```

In der tatsächlichen Implementierung werden in jedem der drei Fälle (A, B, C) noch je zwei weitere Wägungen durchgeführt. Aus den jeweils möglichen Kombinationen von „links schwerer“, „rechts schwerer“ und „gleich“ wird dann abgeleitet, welche Münze gefälscht ist und ob sie leichter oder schwerer als die Normalmünzen ist.

Die vollständige, ausführlich kommentierte Version der Methode kann im Anhang oder direkt im Quelltext nachgelesen werden. Für das Verständnis der objektorientierten Struktur ist vor allem wichtig, dass `FixedTreeStrategy` das Interface `CoinWeighingStrategy` implementiert und damit jederzeit durch eine andere Strategieklassse ausgetauscht werden kann.

4 Vergleichsstrategie RandomGuessStrategy

Um die Qualität einer durchdachten Strategie besser einschätzen zu können, ist es hilfreich, sie mit einem sehr einfachen, nahezu "dummen" Ansatz zu vergleichen. Dazu dient die Klasse `RandomGuessStrategy`, die das Interface `CoinWeighingStrategy` ebenfalls implementiert.

4.1 Idee der Zufallsstrategie

Die `RandomGuessStrategy` ignoriert die Gewichte der Münzen vollständig und führt *keine* Wägungen durch. Stattdessen wird einfach zufällig

- ein Münzindex zwischen 0 und 11 ausgewählt sowie

- zufällig entschieden, ob diese Münze als „schwerer“ oder „leichter“ geraten wird.

Die Trefferwahrscheinlichkeit dieser Strategie ist theoretisch leicht abzuschätzen: Es gibt 12 mögliche Münzen und zu jeder Münze zwei mögliche Zustände (schwerer/leichter), also insgesamt 24 gleich wahrscheinliche Fälle. Zufälliges Raten führt somit im Erwartungswert zu einer Trefferquote von etwa

$$\frac{1}{12 \cdot 2} = \frac{1}{24} \approx 4,17\%.$$

Diese Quote kann in der Simulation als „Basislinie“ dienen, um zu beurteilen, wie gut eine durchdachte Strategie wie `FixedTreeStrategy` tatsächlich ist.

4.2 Implementierung von `RandomGuessStrategy`

Die Implementierung ist sehr kurz und nutzt lediglich ein `Random`-Objekt, um Index und Richtung zu raten:

Listing 5: Zufallsstrategie `RandomGuessStrategy`

```

1  import java.io.PrintWriter;
2  import java.util.Random;
3
4  public class RandomGuessStrategy implements CoinWeighingStrategy {
5
6      private final Random random = new Random();
7
8      @Override
9      public boolean findFakeCoin(int[] coins, int fakeIndex,
10      PrintWriter writer) {
11          // Zufällige Wahl einer Münze 0..11
12          int guessedIndex = random.nextInt(coins.length);
13
14          // Zufällige Annahme: schwerer oder leichter
15          boolean guessedHeavier = random.nextBoolean();
16
17          // Tatsächlicher Zustand der Falschmünze
18          boolean isHeavier = coins[fakeIndex] == 12;
19
20          if (writer != null) {
21              writer.println("RandomGuessStrategy:");
22              writer.println(" geraten: Münze " + (guessedIndex + 1) +
23              " ist " + (guessedHeavier ? "schwerer" : "leichter"));
24              writer.println(" tatsächlich: Münze " + (fakeIndex + 1) +
25              " ist " + (isHeavier ? "schwerer" : "leichter"));
26          }
27
28          // Erfolg nur, wenn sowohl Index als auch Richtung stimmen
29          return guessedIndex == fakeIndex && guessedHeavier == isHeavier;
30      }
31  }

```

4.3 Einsatz in der Simulation

Der große Vorteil der Verwendung des Interfaces `CoinWeighingStrategy` zeigt sich daran, dass die Simulation nicht angepasst werden muss, um eine andere Strategie zu testen. Es reicht, die Instanziierung in der Simulationsklasse anzupassen:

Listing 6: Austausch der Strategie in der Simulation

```

1 // bisher:
2 CoinWeighingStrategy strategy = new FixedTreeStrategy();
3
4 // zum Testen der Zufallsstrategie:
5 CoinWeighingStrategy strategy = new RandomGuessStrategy();

```

Lässt man die Simulation mit `RandomGuessStrategy` über viele Durchläufe laufen (z.B. 1200 oder 10 000), sollte die gemessene Genauigkeit ungefähr bei 4–5 % liegen. Eine deutlich höhere Quote bei `FixedTreeStrategy` zeigt dann, dass die fest implementierte Entscheidungslogik wesentlich besser ist als reines Raten.

In der nächsten Section werden einfache Tests vorgestellt, mit denen überprüft werden kann, ob eine Strategie wie `FixedTreeStrategy` wirklich in allen 24 möglichen Fällen (12 Münzen, jeweils schwerer oder leichter) korrekt arbeitet.

5 Tests der fest verdrahteten Strategie

Damit eine komplexe Entscheidungslogik wie in `FixedTreeStrategy` vertrauenswürdig ist, sollte sie systematisch getestet werden. Im 12-Münzen-Problem gibt es insgesamt nur endlich viele Grundfälle:

- 12 mögliche Positionen der Falschmünze (Münze 1 bis 12),
- zu jeder Münze zwei mögliche Zustände: „schwerer“ oder „leichter“.

Damit ergeben sich $12 \cdot 2 = 24$ unterschiedliche Konstellationen, die eine korrekte Strategie alle richtig behandeln muss. Eine einfache und sehr aussagekräftige Testidee ist daher: *alle 24 Fälle genau einmal durchspielen* und prüfen, ob `findFakeCoin` jeweils `true` zurückgibt.

5.1 Einfache Testklasse ohne JUnit

Die folgende Klasse `FixedTreeStrategyTest` enthält eine `main`-Methode, die alle 24 Grundfälle systematisch testet. Die Ausgaben erfolgen auf der Konsole; für die Tests wird kein `PrintWriter` benötigt, daher wird `null` übergeben.

Listing 7: Systematischer Test aller 24 Fälle

```

1 import java.io.PrintWriter;
2
3 public class FixedTreeStrategyTest {
4
5     public static void main(String[] args) {
6         CoinWeighingStrategy strategy = new FixedTreeStrategy();
7
8         int fehler = 0;
9
10        // Für alle 12 Münzen
11        for (int fakeIndex = 0; fakeIndex < 12; fakeIndex++) {
12            // Zwei Fälle: einmal schwerer, einmal leichter
13            for (boolean heavier : new boolean[]{true, false}) {
14                // Münz-Array vorbereiten
15                int[] coins = new int[12];
16                for (int i = 0; i < 12; i++) {
17                    coins[i] = 10; // normale Münze
18                }
19                coins[fakeIndex] = heavier ? 12 : 8; // Falschmünze setzen

```

```

20
21         // Kein Logging nötig -> writer = null
22         boolean korrekt = strategy.findFakeCoin(coins,
23         fakeIndex,
24         (PrintWriter) null);
25
26         if (!korrekt) {
27             fehler++;
28             System.out.println("FEHLER: Strategie scheitert bei Fall:
29             " +
30             "fakeIndex=" + fakeIndex +
31             " (Münze " + (fakeIndex + 1) + ") " +
32             ", Zustand=" + (heavier ? "schwerer" : "leichter"));
33         }
34     }
35
36     if (fehler == 0) {
37         System.out.println("Alle 24 Fälle korrekt gelöst.");
38     } else {
39         System.out.println("Anzahl fehlerhafter Fälle: " + fehler);
40     }
41 }
42

```

Wenn diese Testklasse meldet, dass alle 24 Fälle korrekt gelöst wurden, ist sichergestellt, dass die aktuelle Implementierung von `FixedTreeStrategy` jede mögliche Position und Richtung der Falschmünze korrekt behandelt (unter der Annahme, dass die Simulationslogik selbst fehlerfrei ist).

Treten Fehler auf, zeigt die Ausgabe genau an, bei welcher Münzposition und welchem Zustand (schwerer/leichter) die Strategie versagt hat. Dies erleichtert das gezielte Debuggen einzelner Zweige im Entscheidungsbaum.

5.2 Hinweis auf JUnit-Tests

In professionelleren Java-Projekten werden Tests häufig mit einem Test-Framework wie JUnit geschrieben. Die oben gezeigte Logik kann leicht in einen JUnit-Test überführt werden. Der folgende Code zeigt eine mögliche Variante mit JUnit 5 (vereinfacht, ohne Import-Details wie `@BeforeEach` etc.):

Listing 8: JUnit-Test für alle 24 Fälle

```

1  import org.junit.jupiter.api.Test;
2  import static org.junit.jupiter.api.Assertions.*;
3
4  public class FixedTreeStrategyJUnitTest {
5
6      @Test
7      void testAll24Cases() {
8          CoinWeighingStrategy strategy = new FixedTreeStrategy();
9
10         for (int fakeIndex = 0; fakeIndex < 12; fakeIndex++) {
11             for (boolean heavier : new boolean[]{true, false}) {
12                 int[] coins = new int[12];
13                 for (int i = 0; i < 12; i++) {
14                     coins[i] = 10;
15                 }
16                 coins[fakeIndex] = heavier ? 12 : 8;

```

```

17         boolean korrekt = strategy.findFakeCoin(coins,
18         fakeIndex,
19         null);
20
21         assertTrue(korrekt,
22         () -> "Fehler bei fakeIndex=" + fakeIndex +
23         " (Münze " + (fakeIndex + 1) + ")" +
24         ", Zustand=" + (heavier ? "schwerer" : "leichter"));
25     }
26 }
27 }
28 }
29 }

```

Dieser Test schlägt automatisch fehl, wenn auch nur eine der 24 Konfigurationen nicht korrekt erkannt wird. Der Fehlermeldungstext enthält dabei direkt die betroffene Münznummer und den Zustand, was die Fehlersuche im Entscheidungsbaum von `FixedTreeStrategy` deutlich vereinfacht.

Mit diesen Tests lässt sich die Korrektheit der Strategie unabhängig von der Simulationsschleife und der Dateiausgabe überprüfen.

6 Verbesserung der Lesbarkeit mit `enum WeighResult`

In der bisherigen Implementierung von `FixedTreeStrategy` verwendet die Hilfsmethode `weigh` einen ganzzahligen Rückgabewert:

- -1 bedeutet: linke Seite ist leichter (rechte schwerer),
- 0 bedeutet: beide Seiten sind gleich schwer,
- 1 bedeutet: linke Seite ist schwerer.

Diese Kodierung ist kompakt, aber fehleranfällig: An verschiedenen Stellen im Code muss man sich merken, wofür -1, 0 und 1 jeweils stehen. Ein vertauschtes Vorzeichen oder ein Missverständnis führt schnell zu falschen Schlussfolgerungen in der Entscheidungslogik.

Eine deutlich lesbarere und sicherere Variante besteht darin, einen *Enum-Typ* zu verwenden, der die drei möglichen Ergebnisse einer Wägung ausdrücklich benennt.

6.1 Definition des Enums `WeighResult`

Der Enum-Typ `WeighResult` enthält genau drei Werte, die die möglichen Ergebnisse einer Wägung beschreiben:

Listing 9: Enum-Typ `WeighResult`

```

1 public enum WeighResult {
2     LEFT_HEAVIER, // linke Seite schwerer
3     BALANCED, // beide Seiten gleich schwer
4     RIGHT_HEAVIER // rechte Seite schwerer
5 }

```

Im Vergleich zur Kodierung mit Ganzzahlen ist sofort erkennbar, was jeder Zustand bedeutet. Dies reduziert Missverständnisse und macht den Code selbst-dokumentierend.

6.2 Angepasste Hilfsmethode weigh

Die Hilfsmethode in `FixedTreeStrategy` kann nun so angepasst werden, dass sie statt eines `int` direkt einen `WeighResult` zurückgibt:

Listing 10: weigh-Methode mit WeighResult

```
1 private WeighResult weigh(int[] coins, int[] left, int[] right) {
2     int leftSum = 0, rightSum = 0;
3     for (int i : left) {
4         leftSum += coins[i];
5     }
6     for (int i : right) {
7         rightSum += coins[i];
8     }
9
10    if (leftSum > rightSum) {
11        return WeighResult.LEFT_HEAVIER;
12    } else if (leftSum < rightSum) {
13        return WeighResult.RIGHT_HEAVIER;
14    } else {
15        return WeighResult.BALANCED;
16    }
17 }
```

Anstelle von -1, 0 und 1 werden nun klar benannte Enum-Konstanten zurückgegeben.

6.3 Verwendung des Enums in der Strategielogik

Auch die Strategiemethode `findFakeCoin` wird dadurch lesbarer. Statt Vergleichsausdrücken wie `if (result1 == 1) {...}` kann direkt auf die Enum-Werte geprüft werden:

Listing 11: Ausschnitt aus `findFakeCoin` mit `WeighResult`

```
1 @Override
2 public boolean findFakeCoin(int[] coins, int fakeCoin, PrintWriter
3     writer) {
4     int identifiedCoin = -1;
5     boolean identifiedHeavier = false;
6
7     int[] left1 = {0, 1, 2, 3};
8     int[] right1 = {4, 5, 6, 7};
9     WeighResult result1 = weigh(coins, left1, right1);
10
11    if (writer != null) {
12        writer.println("Wägung 1: Münzen 1-4 vs. 5-8: " +
13            (result1 == WeighResult.BALANCED ? "gleich"
14            : (result1 == WeighResult.LEFT_HEAVIER
15            ? "1-4 schwerer" : "5-8 schwerer")));
16    }
17
18    if (result1 == WeighResult.BALANCED) {
19        // Fall A: 1-8 sind normal, Falschmünze ist in 9-12
20        //...
21    } else if (result1 == WeighResult.LEFT_HEAVIER) {
22        // Fall B: 1-4 schwerer als 5-8
23        //...
24    } else { // WeighResult.RIGHT_HEAVIER
25        // Fall C: 5-8 schwerer als 1-4
26        //...
```

```

26     }
27
28     boolean isHeavier = coins[fakeCoin] == 12;
29     return identifiedCoin == fakeCoin && identifiedHeavier ==
        isHeavier;
30 }

```

An vielen Stellen im Entscheidungsbaum werden nun statt numerischer Vergleiche Ausdrücke der Form `if (result2 == WeighResult.LEFT_HEAVIER) {... }` oder `else if (result2 == WeighResult.BALANCED) {... }` stehen. Dies macht sofort klar, welcher Waagenzustand gerade betrachtet wird.

6.4 Vorteile des Enum-Ansatzes

Die Verwendung von `WeighResult` bietet mehrere Vorteile:

- **Lesbarkeit:** Statt abstrakter Zahlenwerte wie `-1` oder `1` erscheinen im Code sprechende Namen wie `LEFT_HEAVIER` und `RIGHT_HEAVIER`.
- **Weniger Fehlerquellen:** Die Gefahr, Vorzeichen zu verwechseln (z.B. „-1 bedeutet links schwerer“ statt rechts schwerer), wird deutlich reduziert.
- **Einfache Erweiterbarkeit:** Falls die Waage später um weitere Zustände (z.B. Fehlerfälle) ergänzt werden soll, kann der Enum-Typ leicht erweitert werden, ohne das Interface zu ändern.
- **Bessere Unterstützung durch Werkzeuge:** Moderne IDEs bieten bei Enums Autovervollständigung und Warnungen, wenn nicht alle Enum-Werte in einer `switch`-Anweisung behandelt werden. Das unterstützt eine robuste Implementierung der Entscheidungslogik.

Insgesamt trägt der Einsatz von `enum WeighResult` dazu bei, `FixedTreeStrategy` besser wartbar und verständlicher zu machen, insbesondere wenn der Entscheidungsbaum später weiter verfeinert oder korrigiert werden soll.

7 Zusammenfassung und Ausblick

In diesem Dokument wurde das klassische 12-Münzen-Problem mit Hilfe von Java-Programmen modelliert und untersucht. Ausgehend von der mathematischen Fragestellung (eine von zwölf Münzen ist gefälscht und kann schwerer oder leichter sein, es stehen drei Wägungen zur Verfügung) wurden folgende Schwerpunkte gesetzt:

- Aufbau einer *Simulationsumgebung*, die zufällige Konstellationen erzeugt und die Trefferquote einer Strategie misst,
- Einführung eines *Strategy-Interfaces* (`CoinWeighingStrategy`), um die eigentliche Entscheidungslogik klar von der Simulation zu trennen,
- Implementierung einer fest verdrahteten Entscheidungsstrategie (`FixedTreeStrategy`), die versucht, alle Fälle mit drei Wägungen abzudecken,
- Implementierung einer einfachen Zufallsstrategie (`RandomGuessStrategy`) als Vergleichsmodell,
- systematische Tests (`FixedTreeStrategyTest`), die alle 24 Grundfälle (12 Münzen, jeweils schwerer oder leichter) durchlaufen und damit die Korrektheit einer Strategie prüfen,

- Verbesserung der Lesbarkeit und Wartbarkeit durch den Enum-Typ `WeighResult` zur Darstellung der drei möglichen Wägeergebnisse.

Durch die klare Trennung von Simulation und Strategie lässt sich das Programm leicht erweitern: Neue Strategien müssen nur das Interface `CoinWeighingStrategy` implementieren und können dann direkt in der bestehenden Simulationsumgebung getestet und mit anderen Strategien verglichen werden.

Mögliche Erweiterungen

Das vorgestellte Projekt bietet mehrere natürliche Ansatzpunkte für weitere Experimente und Verbesserungen:

- **Verfeinerung der Entscheidungslogik:** Die `FixedTreeStrategy` kann weiter analysiert und gegebenenfalls so angepasst werden, dass wirklich alle 24 Fälle garantiert korrekt erkannt werden. Die im Anhang vorgestellte Testklasse liefert dabei unmittelbares Feedback.
- **Refactoring des Entscheidungsbaums:** Die teilweise umfangreichen `if`-Verzweigungen könnten mit Hilfe von Hilfsmethoden, Enums und eventuell kleinen Zustandsklassen (z.B. Kandidatenmengen für „schwerer“ und „leichter“) strukturiert und übersichtlicher gestaltet werden.
- **Verwendung von Test-Frameworks:** Anstelle einer eigenen Testklasse könnten die systematischen Tests mit einem Framework wie JUnit implementiert werden. Dies erleichtert die Integration in größere Java-Projekte und automatisierte Testläufe.
- **Verallgemeinerung des Problems:** Statt nur zwölf Münzen könnten Varianten mit anderen Münzzahlen oder anderen Beschränkungen (z.B. nur zwei Wägungen, mehrere falsche Münzen) untersucht werden. Hier bietet sich auch an, die Strategien generischer zu formulieren.
- **Kombination mit Zahlentheorie:** Die im Rahmen anderer Aufgaben verwendeten Konzepte (z.B. ggT-Berechnung, Primfaktorzerlegung) könnten in eigene Utility-Klassen ausgelagert und für weitere mathematische Experimente genutzt werden (z.B. Bruchklassen, Zahlentheorie-Bibliothek).

Insgesamt zeigt das Projekt, wie sich ein klassisches Rätsel nicht nur mathematisch, sondern auch programmiertechnisch interessant aufbereiten lässt. Durch objektorientiertes Design, klar definierte Interfaces und systematische Tests entsteht ein flexibles Gerüst, in dem weitere Strategien und Varianten des Problems leicht erforscht werden können.

A Vollständiger Quelltext

In diesem Anhang sind alle im Dokument verwendeten Java-Klassen vollständig aufgelistet. Die Klassen sind nach ihrer Funktion geordnet:

- `CoinWeighingStrategy`: Interface für Strategien,
- `FixedTreeStrategy`: fest verdrahtete Entscheidungslogik,
- `RandomGuessStrategy`: Zufallsstrategie zum Vergleich,
- `CoinWeighingProblemWithStrategy`: Simulationsprogramm,
- `FixedTreeStrategyTest`: einfacher Test aller 24 Fälle.

A.1 Interface CoinWeighingStrategy

Listing 12: Interface CoinWeighingStrategy

```
1 public interface CoinWeighingStrategy {
2     /**
3      * Versucht, die gefälschte Münze zu finden.
4      *
5      * @param coins Array mit Münzgewichten
6      * @param fakeIndex Index der tatsächlichen Falschmünze (für die
7      *   Auswertung)
8      * @param writer für Protokollausgaben (kann auch null sein)
9      * @return true, wenn Münze und "schwerer/leichter" korrekt
10     * identifiziert wurden
11     */
12     boolean findFakeCoin(int[] coins, int fakeIndex,
13         java.io.PrintWriter writer);
14 }
```

A.2 Enum WeighResult (optionale Verbesserung)

Der folgende Enum-Typ wird verwendet, wenn die Wäageergebnisse nicht mehr als Ganzzahlen -1, 0, 1, sondern als sprechende Konstanten dargestellt werden sollen (vgl. Abschnitt zu WeighResult).

Listing 13: Enum WeighResult

```
1 public enum WeighResult {
2     LEFT_HEAVIER, // linke Seite schwerer
3     BALANCED, // beide Seiten gleich schwer
4     RIGHT_HEAVIER // rechte Seite schwerer
5 }
```

A.3 Simulationsklasse CoinWeighingProblemWithStrategy

Listing 14: Simulationsprogramm CoinWeighingProblemWithStrategy

```
1 import java.io.FileWriter;
2 import java.io.IOException;
3 import java.io.PrintWriter;
4 import java.util.Random;
5
6 public class CoinWeighingProblemWithStrategy {
7
8     public static void main(String[] args) {
9         try (PrintWriter writer = new PrintWriter(new
10             FileWriter("output.txt"))) {
11             writer.println("Ziel ist es, eine gefälschte Münze zu finden.");
12             writer.println("Eine von 12 Münzen ist entweder zu leicht oder
13                 zu schwer. " +
14                 "Man darf nur dreimal wiegen.\n");
15
16             int simulationen = 1200;
17             int richtigeIdentifikationen = 0;
18
19             Random rand = new Random();
20             CoinWeighingStrategy strategy = new FixedTreeStrategy();
```

```

19      // Zum Vergleich mit Zufallsraten:
20      // CoinWeighingStrategy strategy = new RandomGuessStrategy();
21
22      for (int sim = 0; sim < simulationen; sim++) {
23          int[] coins = new int[12];
24          for (int i = 0; i < 12; i++) {
25              coins[i] = 10; // echte Münzen
26          }
27
28          int fakeCoin = rand.nextInt(12); // Index 0..11
29          boolean isHeavier = rand.nextBoolean();
30          coins[fakeCoin] = isHeavier ? 12 : 8; // Falschmünze setzen
31
32          writer.println("\nSimulation gestartet. Eine Münze ist
33                          gefälscht.");
34          writer.println("\t(Für Testzwecke: Münze " + (fakeCoin + 1) +
35                          " ist " +
36                          (isHeavier ? "schwerer (12g)" : "leichter (8g)") + "));
37
38          boolean korrekt = strategy.findFakeCoin(coins, fakeCoin,
39              writer);
40          if (korrekt) {
41              richtigeIdentifikationen++;
42          }
43
44          writer.println();
45          writer.println("Korrekte Identifikationen: " +
46                          richtigeIdentifikationen);
47          writer.println("Genauigkeit: " +
48                          (100.0 * richtigeIdentifikationen / simulationen) + "%");
49      } catch (IOException e) {
50          e.printStackTrace();
51      }
52  }
53  }

```

A.4 Strategieklassse FixedTreeStrategy

Die Klasse FixedTreeStrategy implementiert die fest verdrahtete Entscheidungslogik für das 12-Münzen-Problem. Die Implementierung orientiert sich eng an der im Text diskutierten Version und verwendet eine Hilfsmethode `weigh`, die die beiden Waagschalen vergleicht.

Listing 15: Fest verdrahtete Strategie FixedTreeStrategy

```

1  import java.io.PrintWriter;
2
3  public class FixedTreeStrategy implements CoinWeighingStrategy {
4
5      /**
6       * Vergleicht die Summe zweier Münzgruppen.
7       *
8       * @return -1, wenn links leichter, 0 bei Gleichgewicht,
9       *         1, wenn links schwerer ist
10     */
11     private int weigh(int[] coins, int[] left, int[] right) {
12         int leftSum = 0, rightSum = 0;
13         for (int i : left) {

```



```

14     leftSum += coins[i];
15 }
16 for (int i : right) {
17     rightSum += coins[i];
18 }
19 return Integer.compare(leftSum, rightSum); // -1, 0, 1
20 }
21
22 @Override
23 public boolean findFakeCoin(int[] coins, int fakeCoin, PrintWriter
    writer) {
24     int identifiedCoin = -1; // Index der identifizierten Münze
25     boolean identifiedHeavier = false; // ist sie schwerer?
26
27     // 1. Wägung: Münzen 1-4 vs 5-8
28     int[] left1 = {0, 1, 2, 3};
29     int[] right1 = {4, 5, 6, 7};
30     int result1 = weigh(coins, left1, right1);
31
32     if (writer != null) {
33         writer.println("Wägung 1: Münzen 1-4 vs. 5-8: " +
34             (result1 == 0 ? "gleich"
35             : (result1 == 1 ? "1-4 schwerer" : "5-8 schwerer")));
36     }
37
38     // === Fall A: 1-8 sind gleich schwer -> Falschmünze ist in 9-12
39     ===
40     if (result1 == 0) {
41         int[] left2 = {8, 9, 10}; // Münzen 9-11
42         int[] right2 = {0, 1, 2}; // Münzen 1-3 (als Referenz)
43         int result2 = weigh(coins, left2, right2);
44
45         if (writer != null) {
46             writer.println("Wägung 2: Münzen 9-11 vs. 1-3: " +
47                 (result2 == 0 ? "gleich"
48                 : (result2 == 1 ? "9-11 schwerer" : "9-11 leichter")));
49         }
50
51         if (result2 == 0) {
52             // 9-11 sind normal -> 12 (Index 11) ist falsch
53             identifiedCoin = 11;
54             // Simulations-Trick: direktes Nachschauen im Array
55             identifiedHeavier = coins[11] == 12;
56
57             if (writer != null) {
58                 writer.println("Wägung 3: Münze 1 vs. 12: " +
59                     (coins[11] == 12 ? "12 schwerer" : "1 schwerer"));
60                 writer.println("Ergebnis: Münze 12 ist " +
61                     (coins[11] == 12 ? "schwerer" : "leichter"));
62             }
63
64         } else if (result2 == 1) {
65             // 9-11 insgesamt schwerer -> eine von 9-11 ist schwerer
66             int result3 = weigh(coins, new int[]{8}, new int[]{9}); // 9
67             vs 10
68
69             if (writer != null) {
70                 writer.println("Wägung 3: Münze 9 vs. 10: " +

```

```

69         (result3 == 0 ? "gleich"
70         : (result3 == 1 ? "9 schwerer" : "10 schwerer"))));
71     }
72
73     if (result3 == 0) {
74         identifiedCoin = 10; // Münze 11
75         identifiedHeavier = true;
76         if (writer != null) writer.println("Ergebnis: Münze 11 ist
77             schwerer");
78     } else if (result3 == 1) {
79         identifiedCoin = 8; // Münze 9
80         identifiedHeavier = true;
81         if (writer != null) writer.println("Ergebnis: Münze 9 ist
82             schwerer");
83     } else {
84         identifiedCoin = 9; // Münze 10
85         identifiedHeavier = true;
86         if (writer != null) writer.println("Ergebnis: Münze 10 ist
87             schwerer");
88     }
89
90     } else { // result2 == -1 -> 9-11 insgesamt leichter
91         int result3 = weigh(coins, new int[]{8}, new int[]{9}); // 9
92             vs 10
93
94         if (writer != null) {
95             writer.println("Wägung 3: Münze 9 vs. 10: " +
96             (result3 == 0 ? "gleich"
97             : (result3 == 1 ? "9 schwerer" : "10 schwerer"))));
98         }
99
100        if (result3 == 0) {
101            identifiedCoin = 10; // Münze 11
102            identifiedHeavier = false;
103            if (writer != null) writer.println("Ergebnis: Münze 11 ist
104                leichter");
105        } else if (result3 == 1) {
106            identifiedCoin = 9; // Münze 10
107            identifiedHeavier = false;
108            if (writer != null) writer.println("Ergebnis: Münze 10 ist
109                leichter");
110        } else {
111            identifiedCoin = 8; // Münze 9
112            identifiedHeavier = false;
113            if (writer != null) writer.println("Ergebnis: Münze 9 ist
114                leichter");
115        }
116
117        // === Fall B: 1-4 schwerer als 5-8 ===
118        } else if (result1 == 1) {
119            int[] left2 = {0, 1, 4}; // 1,2,5
120            int[] right2 = {2, 7, 9}; // 3,8,10
121            int result2 = weigh(coins, left2, right2);
122
123            if (writer != null) {
124                writer.println("Wägung 2: Münzen 1,2,5 vs. 3,8,10: " +
125                (result2 == 0 ? "gleich"
126                : (result2 == 1 ? "1,2,5 schwerer" : "1,2,5 leichter"))));
127            }
128        }
129    }

```

```

120     }
121
122     if (result2 == 0) {
123         int result3 = weigh(coins, new int[]{5}, new int[]{6}); //
124         6 vs 7
125
126         if (writer != null) {
127             writer.println("Wägung 3: Münze 6 vs. 7: " +
128                 (result3 == 0 ? "gleich"
129                 : (result3 == 1 ? "6 schwerer" : "7 schwerer")));
130         }
131
132         if (result3 == 1) {
133             identifiedCoin = 6; // Münze 7
134             identifiedHeavier = false; // laut ursprünglicher Logik:
135             7 leichter
136             if (writer != null) writer.println("Ergebnis: Münze 7 ist
137                 leichter");
138         } else if (result3 == -1) {
139             identifiedCoin = 5; // Münze 6
140             identifiedHeavier = false;
141             if (writer != null) writer.println("Ergebnis: Münze 6 ist
142                 leichter");
143         } else {
144             identifiedCoin = 3; // Münze 4
145             identifiedHeavier = true;
146             if (writer != null) writer.println("Ergebnis: Münze 4 ist
147                 schwerer");
148         }
149     } else if (result2 == 1) {
150         int result3 = weigh(coins, new int[]{0}, new int[]{1}); //
151         1 vs 2
152
153         if (writer != null) {
154             writer.println("Wägung 3: Münze 1 vs. 2: " +
155                 (result3 == 0 ? "gleich"
156                 : (result3 == 1 ? "1 schwerer" : "2 schwerer")));
157         }
158
159         if (result3 == 0) {
160             identifiedCoin = 7; // Münze 8
161             identifiedHeavier = false;
162             if (writer != null) writer.println("Ergebnis: Münze 8 ist
163                 leichter");
164         } else if (result3 == 1) {
165             identifiedCoin = 0; // Münze 1
166             identifiedHeavier = true;
167             if (writer != null) writer.println("Ergebnis: Münze 1 ist
168                 schwerer");
169         } else {
170             identifiedCoin = 1; // Münze 2
171             identifiedHeavier = true;
172             if (writer != null) writer.println("Ergebnis: Münze 2 ist
173                 schwerer");
174         }
175     } else { // result2 == -1

```

```

169         int result3 = weigh(coins, new int[]{2}, new int[]{0}); //
           3 vs 1
170
171         if (writer != null) {
172             writer.println("Wägung 3: Münze 3 vs. 1: " +
173                 (result3 == 0 ? "gleich"
174                 : (result3 == 1 ? "3 schwerer" : "1 schwerer")));
175         }
176
177         if (result3 == 1) {
178             identifiedCoin = 2; // Münze 3
179             identifiedHeavier = true;
180             if (writer != null) writer.println("Ergebnis: Münze 3 ist
           schwerer");
181         } else if (result3 == -1) {
182             identifiedCoin = 0; // Münze 1
183             identifiedHeavier = false;
184             if (writer != null) writer.println("Ergebnis: Münze 1 ist
           leichter");
185         } else {
186             identifiedCoin = 4; // Münze 5
187             identifiedHeavier = false;
188             if (writer != null) writer.println("Ergebnis: Münze 5 ist
           leichter");
189         }
190
191         // == Fall C: 5-8 schwerer als 1-4 ==
192     } else { // result1 == -1
193         int[] left2 = {4, 5, 0}; // 5,6,1
194         int[] right2 = {3, 7, 9}; // 4,8,10
195         int result2 = weigh(coins, left2, right2);
196
197         if (writer != null) {
198             writer.println("Wägung 2: Münzen 5,6,1 vs. 4,8,10: " +
199                 (result2 == 0 ? "gleich"
200                 : (result2 == 1 ? "5,6,1 schwerer" : "5,6,1 leichter")));
201         }
202
203         if (result2 == 0) {
204             int result3 = weigh(coins, new int[]{1}, new int[]{2});
           // 2 vs 3
205
206             if (writer != null) {
207                 writer.println("Wägung 3: Münze 2 vs. 3: " +
208                     (result3 == 0 ? "gleich"
209                     : (result3 == 1 ? "2 schwerer" : "3 schwerer")));
210             }
211
212             if (result3 == 0) {
213                 identifiedCoin = 6; // Münze 7
214                 identifiedHeavier = true;
215                 if (writer != null) writer.println("Ergebnis: Münze 7
           ist schwerer");
216             } else if (result3 == 1) {
217                 identifiedCoin = 2; // Münze 3
218                 identifiedHeavier = false;
219                 if (writer != null) writer.println("Ergebnis: Münze 3
           ist leichter");

```

```

220     } else {
221         identifiedCoin = 1; // Münze 2
222         identifiedHeavier = false;
223         if (writer != null) writer.println("Ergebnis: Münze 2
           ist leichter");
224     }
225
226 } else if (result2 == 1) {
227     int result3 = weigh(coins, new int[]{4}, new int[]{5});
           // 5 vs 6
228
229     if (writer != null) {
230         writer.println("Wägung 3: Münze 5 vs. 6: " +
231             (result3 == 0 ? "gleich"
232             : (result3 == 1 ? "5 schwerer" : "6 schwerer")));
233     }
234
235     if (result3 == 0) {
236         identifiedCoin = 3; // Münze 4
237         identifiedHeavier = false;
238         if (writer != null) writer.println("Ergebnis: Münze 4
           ist leichter");
239     } else if (result3 == 1) {
240         identifiedCoin = 4; // Münze 5
241         identifiedHeavier = true;
242         if (writer != null) writer.println("Ergebnis: Münze 5
           ist schwerer");
243     } else {
244         identifiedCoin = 5; // Münze 6
245         identifiedHeavier = true;
246         if (writer != null) writer.println("Ergebnis: Münze 6
           ist schwerer");
247     }
248
249 } else { // result2 == -1
250     int result3 = weigh(coins, new int[]{6}, new int[]{7});
           // 7 vs 8
251
252     if (writer != null) {
253         writer.println("Wägung 3: Münze 7 vs. 8: " +
254             (result3 == 0 ? "gleich"
255             : (result3 == 1 ? "7 schwerer" : "8 schwerer")));
256     }
257
258     if (result3 == 0) {
259         identifiedCoin = 0; // Münze 1
260         identifiedHeavier = false;
261         if (writer != null) writer.println("Ergebnis: Münze 1
           ist leichter");
262     } else if (result3 == 1) {
263         identifiedCoin = 6; // Münze 7
264         identifiedHeavier = true;
265         if (writer != null) writer.println("Ergebnis: Münze 7
           ist schwerer");
266     } else {
267         identifiedCoin = 7; // Münze 8
268         identifiedHeavier = true;
269         if (writer != null) writer.println("Ergebnis: Münze 8

```

```

270         }
271     }
272 }
273
274     boolean isHeavier = coins[fakeCoin] == 12;
275     return identifiedCoin == fakeCoin && identifiedHeavier ==
        isHeavier;
276 }
277 }

```

A.5 Alternative Implementierung mit WeighResult

Neben der Variante mit ganzzahligem Rückgabewert in Listing 15 folgt hier eine alternative Implementierung von `FixedTreeStrategy`, die das Enum `WeighResult` (vgl. Listing 13) verwendet. Die fachliche Logik ist dieselbe, die Lesbarkeit wird jedoch verbessert.

Listing 16: `FixedTreeStrategy` mit Enum `WeighResult`

```

1  import java.io.PrintWriter;
2
3  public class FixedTreeStrategy implements CoinWeighingStrategy {
4
5      /**
6       * Vergleicht die Summe zweier Münzgruppen und liefert
7       * ein sprechendes Wägeergebnis zurück.
8       */
9      private WeighResult weigh(int[] coins, int[] left, int[] right) {
10         int leftSum = 0, rightSum = 0;
11         for (int i : left) {
12             leftSum += coins[i];
13         }
14         for (int i : right) {
15             rightSum += coins[i];
16         }
17
18         if (leftSum > rightSum) {
19             return WeighResult.LEFT_HEAVIER;
20         } else if (leftSum < rightSum) {
21             return WeighResult.RIGHT_HEAVIER;
22         } else {
23             return WeighResult.BALANCED;
24         }
25     }
26
27     @Override
28     public boolean findFakeCoin(int[] coins, int fakeCoin,
29     PrintWriter writer) {
30         int identifiedCoin = -1; // Index der identifizierten Münze
31         boolean identifiedHeavier = false; // ist sie schwerer?
32
33         // 1. Wägung: Münzen 1-4 vs 5-8
34         int[] left1 = {0, 1, 2, 3};
35         int[] right1 = {4, 5, 6, 7};
36         WeighResult result1 = weigh(coins, left1, right1);
37
38         if (writer != null) {
39             writer.println("Wägung 1: Münzen 1-4 vs. 5-8: " +

```

```

40     (result1 == WeighResult.BALANCED ? "gleich"
41     : (result1 == WeighResult.LEFT_HEAVIER
42     ? "1-4 schwerer" : "5-8 schwerer"))));
43 }
44
45 // === Fall A: 1-8 sind gleich schwer -> Falschmünze ist in 9-12
46 // ===
47 if (result1 == WeighResult.BALANCED) {
48     int[] left2 = {8, 9, 10}; // Münzen 9-11
49     int[] right2 = {0, 1, 2}; // Münzen 1-3 (Referenz)
50     WeighResult result2 = weigh(coins, left2, right2);
51
52     if (writer != null) {
53         writer.println("Wägung 2: Münzen 9-11 vs. 1-3: " +
54         (result2 == WeighResult.BALANCED ? "gleich"
55         : (result2 == WeighResult.LEFT_HEAVIER
56         ? "9-11 schwerer" : "9-11 leichter"))));
57     }
58
59     if (result2 == WeighResult.BALANCED) {
60         // 9-11 sind normal -> 12 (Index 11) ist falsch
61         identifiedCoin = 11;
62         // Simulations-Trick: direktes Nachschauen
63         identifiedHeavier = coins[11] == 12;
64
65         if (writer != null) {
66             writer.println("Wägung 3: Münze 1 vs. 12: " +
67             (coins[11] == 12 ? "12 schwerer"
68             : "1 schwerer"));
69             writer.println("Ergebnis: Münze 12 ist " +
70             (coins[11] == 12 ? "schwerer" : "leichter"));
71         }
72     } else if (result2 == WeighResult.LEFT_HEAVIER) {
73         // 9-11 insgesamt schwerer -> eine von 9-11 ist schwerer
74         WeighResult result3 = weigh(coins, new int[]{8},
75         new int[]{9}); // 9 vs 10
76
77         if (writer != null) {
78             writer.println("Wägung 3: Münze 9 vs. 10: " +
79             (result3 == WeighResult.BALANCED ? "gleich"
80             : (result3 == WeighResult.LEFT_HEAVIER
81             ? "9 schwerer" : "10 schwerer"))));
82         }
83
84         if (result3 == WeighResult.BALANCED) {
85             identifiedCoin = 10; // Münze 11
86             identifiedHeavier = true;
87             if (writer != null) writer.println("Ergebnis: Münze 11 ist
88             schwerer");
89         } else if (result3 == WeighResult.LEFT_HEAVIER) {
90             identifiedCoin = 8; // Münze 9
91             identifiedHeavier = true;
92             if (writer != null) writer.println("Ergebnis: Münze 9 ist
93             schwerer");
94         } else { // RIGHT_HEAVIER
95             identifiedCoin = 9; // Münze 10
96             identifiedHeavier = true;

```

```

95         if (writer != null) writer.println("Ergebnis: Münze 10 ist
96             schwerer");
97     }
98 } else { // result2 == RIGHT_HEAVIER -> 9-11 insgesamt leichter
99     WeighResult result3 = weigh(coins, new int[]{8},
100         new int[]{9}); // 9 vs 10
101
102     if (writer != null) {
103         writer.println("Wägung 3: Münze 9 vs. 10: " +
104             (result3 == WeighResult.BALANCED ? "gleich"
105             : (result3 == WeighResult.LEFT_HEAVIER
106             ? "9 schwerer" : "10 schwerer")));
107     }
108
109     if (result3 == WeighResult.BALANCED) {
110         identifiedCoin = 10; // Münze 11
111         identifiedHeavier = false;
112         if (writer != null) writer.println("Ergebnis: Münze 11 ist
113             leichter");
114     } else if (result3 == WeighResult.LEFT_HEAVIER) {
115         identifiedCoin = 9; // Münze 10
116         identifiedHeavier = false;
117         if (writer != null) writer.println("Ergebnis: Münze 10 ist
118             leichter");
119     } else { // RIGHT_HEAVIER
120         identifiedCoin = 8; // Münze 9
121         identifiedHeavier = false;
122         if (writer != null) writer.println("Ergebnis: Münze 9 ist
123             leichter");
124     }
125
126     // === Fall B: 1-4 schwerer als 5-8 ===
127 } else if (result1 == WeighResult.LEFT_HEAVIER) {
128     int[] left2 = {0, 1, 4}; // 1,2,5
129     int[] right2 = {2, 7, 9}; // 3,8,10
130     WeighResult result2 = weigh(coins, left2, right2);
131
132     if (writer != null) {
133         writer.println("Wägung 2: Münzen 1,2,5 vs. 3,8,10: " +
134             (result2 == WeighResult.BALANCED ? "gleich"
135             : (result2 == WeighResult.LEFT_HEAVIER
136             ? "1,2,5 schwerer" : "1,2,5 leichter")));
137     }
138
139     if (result2 == WeighResult.BALANCED) {
140         WeighResult result3 = weigh(coins, new int[]{5},
141             new int[]{6}); // 6 vs 7
142
143         if (writer != null) {
144             writer.println("Wägung 3: Münze 6 vs. 7: " +
145                 (result3 == WeighResult.BALANCED ? "gleich"
146                 : (result3 == WeighResult.LEFT_HEAVIER
147                 ? "6 schwerer" : "7 schwerer")));
148         }
149
150         if (result3 == WeighResult.LEFT_HEAVIER) {
151             identifiedCoin = 6; // Münze 7

```



```

149         identifiedHeavier = false; // ursprüngliche Logik: 7
150         leichter
151     if (writer != null) writer.println("Ergebnis: Münze 7 ist
152         leichter");
153 } else if (result3 == WeighResult.RIGHT_HEAVIER) {
154     identifiedCoin = 5; // Münze 6
155     identifiedHeavier = false;
156     if (writer != null) writer.println("Ergebnis: Münze 6 ist
157         leichter");
158 } else { // BALANCED
159     identifiedCoin = 3; // Münze 4
160     identifiedHeavier = true;
161     if (writer != null) writer.println("Ergebnis: Münze 4 ist
162         schwerer");
163 }
164
165 } else if (result2 == WeighResult.LEFT_HEAVIER) {
166     WeighResult result3 = weigh(coins, new int[]{0},
167     new int[]{1}); // 1 vs 2
168
169     if (writer != null) {
170         writer.println("Wägung 3: Münze 1 vs. 2: " +
171         (result3 == WeighResult.BALANCED ? "gleich"
172         : (result3 == WeighResult.LEFT_HEAVIER
173         ? "1 schwerer" : "2 schwerer")));
174     }
175
176     if (result3 == WeighResult.BALANCED) {
177         identifiedCoin = 7; // Münze 8
178         identifiedHeavier = false;
179         if (writer != null) writer.println("Ergebnis: Münze 8 ist
180             leichter");
181     } else if (result3 == WeighResult.LEFT_HEAVIER) {
182         identifiedCoin = 0; // Münze 1
183         identifiedHeavier = true;
184         if (writer != null) writer.println("Ergebnis: Münze 1 ist
185             schwerer");
186     } else { // RIGHT_HEAVIER
187         identifiedCoin = 1; // Münze 2
188         identifiedHeavier = true;
189         if (writer != null) writer.println("Ergebnis: Münze 2 ist
190             schwerer");
191     }
192
193 } else { // result2 == RIGHT_HEAVIER
194     WeighResult result3 = weigh(coins, new int[]{2},
195     new int[]{0}); // 3 vs 1
196
197     if (writer != null) {
198         writer.println("Wägung 3: Münze 3 vs. 1: " +
199         (result3 == WeighResult.BALANCED ? "gleich"
200         : (result3 == WeighResult.LEFT_HEAVIER
201         ? "3 schwerer" : "1 schwerer")));
202     }
203
204     if (result3 == WeighResult.LEFT_HEAVIER) {
205         identifiedCoin = 2; // Münze 3
206         identifiedHeavier = true;

```

```

200         if (writer != null) writer.println("Ergebnis: Münze 3 ist
           schwerer");
201     } else if (result3 == WeighResult.RIGHT_HEAVIER) {
202         identifiedCoin = 0; // Münze 1
203         identifiedHeavier = false;
204         if (writer != null) writer.println("Ergebnis: Münze 1 ist
           leichter");
205     } else { // BALANCED
206         identifiedCoin = 4; // Münze 5
207         identifiedHeavier = false;
208         if (writer != null) writer.println("Ergebnis: Münze 5 ist
           leichter");
209     }
210
211     // === Fall C: 5-8 schwerer als 1-4 ===
212 } else { // result1 == RIGHT_HEAVIER
213     int[] left2 = {4, 5, 0}; // 5,6,1
214     int[] right2 = {3, 7, 9}; // 4,8,10
215     WeighResult result2 = weigh(coins, left2, right2);
216
217     if (writer != null) {
218         writer.println("Wägung 2: Münzen 5,6,1 vs. 4,8,10: " +
219             (result2 == WeighResult.BALANCED ? "gleich"
220             : (result2 == WeighResult.LEFT_HEAVIER
221             ? "5,6,1 schwerer" : "5,6,1 leichter")));
222     }
223
224     if (result2 == WeighResult.BALANCED) {
225         WeighResult result3 = weigh(coins, new int[]{1},
226             new int[]{2}); // 2 vs 3
227
228         if (writer != null) {
229             writer.println("Wägung 3: Münze 2 vs. 3: " +
230                 (result3 == WeighResult.BALANCED ? "gleich"
231                 : (result3 == WeighResult.LEFT_HEAVIER
232                 ? "2 schwerer" : "3 schwerer")));
233         }
234
235         if (result3 == WeighResult.BALANCED) {
236             identifiedCoin = 6; // Münze 7
237             identifiedHeavier = true;
238             if (writer != null) writer.println("Ergebnis: Münze 7
               ist schwerer");
239         } else if (result3 == WeighResult.LEFT_HEAVIER) {
240             identifiedCoin = 2; // Münze 3
241             identifiedHeavier = false;
242             if (writer != null) writer.println("Ergebnis: Münze 3
               ist leichter");
243         } else { // RIGHT_HEAVIER
244             identifiedCoin = 1; // Münze 2
245             identifiedHeavier = false;
246             if (writer != null) writer.println("Ergebnis: Münze 2
               ist leichter");
247         }
248
249     } else if (result2 == WeighResult.LEFT_HEAVIER) {
250         WeighResult result3 = weigh(coins, new int[]{4},
251             new int[]{5}); // 5 vs 6

```

```

252
253         if (writer != null) {
254             writer.println("Wägung 3: Münze 5 vs. 6: " +
255                 (result3 == WeighResult.BALANCED ? "gleich"
256                 : (result3 == WeighResult.LEFT_HEAVIER
257                 ? "5 schwerer" : "6 schwerer")));
258         }
259
260         if (result3 == WeighResult.BALANCED) {
261             identifiedCoin = 3; // Münze 4
262             identifiedHeavier = false;
263             if (writer != null) writer.println("Ergebnis: Münze 4
264                 ist leichter");
265         } else if (result3 == WeighResult.LEFT_HEAVIER) {
266             identifiedCoin = 4; // Münze 5
267             identifiedHeavier = true;
268             if (writer != null) writer.println("Ergebnis: Münze 5
269                 ist schwerer");
270         } else { // RIGHT_HEAVIER
271             identifiedCoin = 5; // Münze 6
272             identifiedHeavier = true;
273             if (writer != null) writer.println("Ergebnis: Münze 6
274                 ist schwerer");
275         }
276
277         } else { // result2 == RIGHT_HEAVIER
278             WeighResult result3 = weigh(coins, new int[]{6},
279                 new int[]{7}); // 7 vs 8
280
281             if (writer != null) {
282                 writer.println("Wägung 3: Münze 7 vs. 8: " +
283                     (result3 == WeighResult.BALANCED ? "gleich"
284                     : (result3 == WeighResult.LEFT_HEAVIER
285                     ? "7 schwerer" : "8 schwerer")));
286             }
287
288             if (result3 == WeighResult.BALANCED) {
289                 identifiedCoin = 0; // Münze 1
290                 identifiedHeavier = false;
291                 if (writer != null) writer.println("Ergebnis: Münze 1
292                     ist leichter");
293             } else if (result3 == WeighResult.LEFT_HEAVIER) {
294                 identifiedCoin = 6; // Münze 7
295                 identifiedHeavier = true;
296                 if (writer != null) writer.println("Ergebnis: Münze 7
297                     ist schwerer");
298             } else { // RIGHT_HEAVIER
299                 identifiedCoin = 7; // Münze 8
300                 identifiedHeavier = true;
301                 if (writer != null) writer.println("Ergebnis: Münze 8
302                     ist schwerer");
303             }
304         }
305     }
306 }
307
308 boolean isHeavier = coins[fakeCoin] == 12;
309 return identifiedCoin == fakeCoin && identifiedHeavier ==
310     isHeavier;

```

```
303     }
304 }
```

A.6 Vergleichsstrategie RandomGuessStrategy

Die Klasse RandomGuessStrategy implementiert eine sehr simple Strategie, die keine Wägungen durchführt, sondern rein zufällig eine Münze und deren Zustand (schwerer/leichter) rät.

Listing 17: Zufallsstrategie RandomGuessStrategy

```
1  import java.io.PrintWriter;
2  import java.util.Random;
3
4  public class RandomGuessStrategy implements CoinWeighingStrategy {
5
6      private final Random random = new Random();
7
8      @Override
9      public boolean findFakeCoin(int[] coins, int fakeIndex,
10      PrintWriter writer) {
11          // Zufällige Wahl einer Münze 0..11
12          int guessedIndex = random.nextInt(coins.length);
13
14          // Zufällige Annahme: schwerer oder leichter
15          boolean guessedHeavier = random.nextBoolean();
16
17          // Tatsächlicher Zustand der Falschmünze
18          boolean isHeavier = coins[fakeIndex] == 12;
19
20          if (writer != null) {
21              writer.println("RandomGuessStrategy:");
22              writer.println(" geraten: Münze " + (guessedIndex + 1) +
23              " ist " + (guessedHeavier ? "schwerer" : "leichter"));
24              writer.println(" tatsächlich: Münze " + (fakeIndex + 1) +
25              " ist " + (isHeavier ? "schwerer" : "leichter"));
26          }
27
28          // Erfolg nur, wenn sowohl Index als auch Richtung stimmen
29          return guessedIndex == fakeIndex && guessedHeavier == isHeavier;
30      }
31 }
```

A.7 Testklasse FixedTreeStrategyTest

Mit der folgenden Testklasse werden alle 24 Grundfälle (12 Münzen, jeweils schwerer oder leichter) systematisch geprüft. Die Klasse kann unabhängig von der Simulation ausgeführt werden und liefert eine einfache Rückmeldung über die Korrektheit der fest verdrahteten Strategie.

Listing 18: Systematischer Test aller 24 Fälle

```
1  import java.io.PrintWriter;
2
3  public class FixedTreeStrategyTest {
4
5      public static void main(String[] args) {
6          CoinWeighingStrategy strategy = new FixedTreeStrategy();
7
8          int fehler = 0;
```

```

9
10 // Für alle 12 Münzen
11 for (int fakeIndex = 0; fakeIndex < 12; fakeIndex++) {
12     // Zwei Fälle: einmal schwerer, einmal leichter
13     for (boolean heavier : new boolean[]{true, false}) {
14         // Münz-Array vorbereiten
15         int[] coins = new int[12];
16         for (int i = 0; i < 12; i++) {
17             coins[i] = 10; // normale Münze
18         }
19         coins[fakeIndex] = heavier ? 12 : 8; // Falschmünze setzen
20
21         // Kein Logging nötig -> writer = null
22         boolean korrekt = strategy.findFakeCoin(coins,
23             fakeIndex,
24             (PrintWriter) null);
25
26         if (!korrekt) {
27             fehler++;
28             System.out.println("FEHLER: Strategie scheitert bei Fall: "
29                 +
30                 "fakeIndex=" + fakeIndex +
31                 " (Münze " + (fakeIndex + 1) + ") " +
32                 ", Zustand=" + (heavier ? "schwerer" : "leichter"));
33         }
34     }
35
36     if (fehler == 0) {
37         System.out.println("Alle 24 Fälle korrekt gelöst.");
38     } else {
39         System.out.println("Anzahl fehlerhafter Fälle: " + fehler);
40     }
41 }
42 }

```

A.8 Alte Implementierung ohne Strategy-Pattern

Die folgende Programmliste zeigt eine frühere Version der Lösung für das 12-Münzen-Problem. In dieser Variante sind Simulation, Entscheidungslogik und Ausgaben noch in einer einzigen Klasse bzw. Methode gebündelt. Es gibt weder ein eigenes Interface für Strategien noch eine klare Trennung zwischen

- der Erzeugung zufälliger Testfälle (Simulation) und
- der eigentlichen Suchstrategie zur Bestimmung der Falschmünze.

Gerade im Vergleich zur später eingeführten Struktur mit `CoinWeighingStrategy`, `FixedTreeStrategy` und `RandomGuessStrategy` wird hier deutlich, welche Vorteile Interfaces und objektorientiertes Design bieten: Strategien können getrennt von der Simulation entwickelt, leichter getestet und untereinander ausgetauscht werden.

Listing 19: Alte Variante ohne Strategy-Interface

```

1
2
3 import java.io.*;

```

```

4 import java.util.Random;
5
6 public class CoinWeighingProblem_mit_Statistik_mit_Ausgabe {
7     public static void main(String[] args) {
8         try (PrintWriter writer = new PrintWriter(new
9             FileWriter("output.txt"))) {
10             writer.println("Ziel ist es, eine gefälschte Münze zu finden.");
11             writer.println("Eine von 12 Münzen ist entweder zu leicht oder zu
12                 schwer. Man darf nur dreimal wiegen.\n");
13
14             int simulationen = 1200; // Anzahl der Simulationen
15             int richtigeIdentifikationen = 0;
16
17             for (int sim = 0; sim < simulationen; sim++) {
18                 int[] coins = new int[12];
19                 for (int i = 0; i < 12; i++) {
20                     coins[i] = 10;
21                 }
22
23                 Random rand = new Random();
24                 int fakeCoin = rand.nextInt(12);
25                 boolean isHeavier = rand.nextBoolean();
26                 coins[fakeCoin] = isHeavier ? 12 : 8;
27                 writer.println("\nSimulation gestartet. Eine Münze ist
28                     gefälscht.");
29                 writer.println("\t(Für Testzwecke: Münze " + (fakeCoin + 1) + "
30                     ist " +
31                     (isHeavier ? "schwerer (12g)" : "leichter (8g)") + ")");
32
33                 // Führe die Wägungen durch
34                 if (findFakeCoin(coins, fakeCoin, writer)) {
35                     richtigeIdentifikationen++;
36                 }
37             }
38
39             writer.println("Korrekte Identifikationen: " +
40                 richtigeIdentifikationen);
41             writer.println("Genauigkeit: " + (100.0 *
42                 richtigeIdentifikationen / simulationen) + "%");
43         } catch (IOException e) {
44             e.printStackTrace();
45         }
46     }
47
48     public static int weigh(int[] coins, int[] left, int[] right) {
49         int leftSum = 0, rightSum = 0;
50         for (int i : left) leftSum += coins[i];
51         for (int i : right) rightSum += coins[i];
52         if (leftSum < rightSum) return -1;
53         if (leftSum > rightSum) return 1;
54         return 0;
55     }
56
57     public static boolean findFakeCoin(int[] coins, int
58         fakeCoin, PrintWriter writer) {
59         int identifiedCoin = -1; // Index der identifizierten Münze
60         boolean identifiedHeavier = false; // Ist die identifizierte Münze
61             schwerer?

```

```

54  int[] left1 = {0, 1, 2, 3};
55  int[] right1 = {4, 5, 6, 7};
56  int result1 = weigh(coins, left1, right1);
57  writer.println("Wägung 1: Münzen 1-4 vs. 5-8: " +
58  (result1 == 0 ? "gleich" : (result1 == 1 ? "1-4 schwerer" : "5-8
    schwerer"))));
59
60  if (result1 == 0) {
61      int[] left2 = {8, 9, 10};
62      int[] right2 = {0, 1, 2};
63      int result2 = weigh(coins, left2, right2);
64      writer.println("Wägung 2: Münzen 9-11 vs. 1-3: " +
65      (result2 == 0 ? "gleich" : (result2 == 1 ? "9-11 schwerer" :
        "9-11 leichter"))));
66
67      if (result2 == 0) {
68          identifiedCoin = 11; // Münze 12
69          identifiedHeavier = coins[11] == 12;
70          writer.println("Wägung 3: Münze 1 vs. 12: " + (coins[11] == 12
            ? "12 schwerer" : "1 schwerer"));
71          writer.println("Ergebnis: Münze 12 ist " + (coins[11] == 12 ?
            "schwerer" : "leichter"));
72      } else if (result2 == 1) {
73          int result3 = weigh(coins, new int[]{8}, new int[]{9});
74          writer.println("Wägung 3: Münze 9 vs. 10: " +
75          (result3 == 0 ? "gleich" : (result3 == 1 ? "9 schwerer" : "10
            schwerer"))));
76          if (result3 == 0) { identifiedCoin = 10; identifiedHeavier =
            true; writer.println("Ergebnis: Münze 11 ist schwerer"); }
77          else if (result3 == 1) { identifiedCoin = 8; identifiedHeavier
            = true; writer.println("Ergebnis: Münze 9 ist schwerer"); }
78          else { identifiedCoin = 9; identifiedHeavier = true;
            writer.println("Ergebnis: Münze 10 ist schwerer"); }
79      } else {
80          int result3 = weigh(coins, new int[]{8}, new int[]{9});
81          writer.println("Wägung 3: Münze 9 vs. 10: " +
82          (result3 == 0 ? "gleich" : (result3 == 1 ? "9 schwerer" : "10
            schwerer"))));
83          if (result3 == 0) { identifiedCoin = 10; identifiedHeavier =
            false; writer.println("Ergebnis: Münze 11 ist leichter"); }
84          else if (result3 == 1) { identifiedCoin = 9; identifiedHeavier
            = false; writer.println("Ergebnis: Münze 10 ist leichter"); }
85          else { identifiedCoin = 8; identifiedHeavier = false;
            writer.println("Ergebnis: Münze 9 ist leichter"); }
86      }
87  } else if (result1 == 1) {
88      int[] left2 = {0, 1, 4};
89      int[] right2 = {2, 7, 9};
90      int result2 = weigh(coins, left2, right2);
91      writer.println("Wägung 2: Münzen 1,2,5 vs. 3,8,10: " +
92      (result2 == 0 ? "gleich" : (result2 == 1 ? "1,2,5 schwerer" :
        "1,2,5 leichter"))));
93
94      if (result2 == 0) {
95          int result3 = weigh(coins, new int[]{5}, new int[]{6});
96          writer.println("Wägung 3: Münze 6 vs. 7: " +
97          (result3 == 0 ? "gleich" : (result3 == 1 ? "6 schwerer" : "7
            schwerer"))));

```

```

98     if (result3 == 1) { identifiedCoin = 6; identifiedHeavier =
99         false; writer.println("Ergebnis: Münze 7 ist leichter"); }
100    else if (result3 == -1) { identifiedCoin = 5; identifiedHeavier
101        = false; writer.println("Ergebnis: Münze 6 ist leichter"); }
102    else { identifiedCoin = 3; identifiedHeavier = true;
103        writer.println("Ergebnis: Münze 4 ist schwerer"); }
104    } else if (result2 == 1) {
105        int result3 = weigh(coins, new int[]{0}, new int[]{1});
106        writer.println("Wägung 3: Münze 1 vs. 2: " +
107            (result3 == 0 ? "gleich" : (result3 == 1 ? "1 schwerer" : "2
108                schwerer"))));
109        if (result3 == 0) { identifiedCoin = 7; identifiedHeavier =
110            false; writer.println("Ergebnis: Münze 8 ist leichter"); }
111        else if (result3 == 1) { identifiedCoin = 0; identifiedHeavier
112            = true; writer.println("Ergebnis: Münze 1 ist schwerer"); }
113        else { identifiedCoin = 1; identifiedHeavier = true;
114            writer.println("Ergebnis: Münze 2 ist schwerer"); }
115    } else {
116        int result3 = weigh(coins, new int[]{2}, new int[]{0});
117        writer.println("Wägung 3: Münze 3 vs. 1: " +
118            (result3 == 0 ? "gleich" : (result3 == 1 ? "3 schwerer" : "1
119                schwerer"))));
120        if (result3 == 1) { identifiedCoin = 2; identifiedHeavier =
121            true; writer.println("Ergebnis: Münze 3 ist schwerer"); }
122        else if (result3 == -1) { identifiedCoin = 0; identifiedHeavier
123            = false; writer.println("Ergebnis: Münze 1 ist leichter"); }
124        else { identifiedCoin = 4; identifiedHeavier = false;
125            writer.println("Ergebnis: Münze 5 ist leichter"); }
126    }
127    } else {
128        int[] left2 = {4, 5, 0};
129        int[] right2 = {3, 7, 9};
130        int result2 = weigh(coins, left2, right2);
131        writer.println("Wägung 2: Münzen 5,6,1 vs. 4,8,10: " +
132            (result2 == 0 ? "gleich" : (result2 == 1 ? "5,6,1 schwerer" :
133                "5,6,1 leichter"))));
134
135        if (result2 == 0) {
136            int result3 = weigh(coins, new int[]{1}, new int[]{2});
137            writer.println("Wägung 3: Münze 2 vs. 3: " +
138                (result3 == 0 ? "gleich" : (result3 == 1 ? "2 schwerer" : "3
139                    schwerer"))));
140            if (result3 == 0) { identifiedCoin = 6; identifiedHeavier =
141                true; writer.println("Ergebnis: Münze 7 ist schwerer"); }
142            else if (result3 == 1) { identifiedCoin = 2; identifiedHeavier
143                = false; writer.println("Ergebnis: Münze 3 ist leichter"); }
144            else { identifiedCoin = 1; identifiedHeavier = false;
145                writer.println("Ergebnis: Münze 2 ist leichter"); }
146        } else if (result2 == 1) {
147            int result3 = weigh(coins, new int[]{4}, new int[]{5});
148            writer.println("Wägung 3: Münze 5 vs. 6: " +
149                (result3 == 0 ? "gleich" : (result3 == 1 ? "5 schwerer" : "6
150                    schwerer"))));
151            if (result3 == 0) { identifiedCoin = 3; identifiedHeavier =
152                false; writer.println("Ergebnis: Münze 4 ist leichter"); }
153            else if (result3 == 1) { identifiedCoin = 4; identifiedHeavier
154                = true; writer.println("Ergebnis: Münze 5 ist schwerer"); }
155            else { identifiedCoin = 5; identifiedHeavier = true;

```



```

137     writer.println("Ergebnis: Münze 6 ist schwerer"); }
138 } else {
139     int result3 = weigh(coins, new int[]{6}, new int[]{7});
140     writer.println("Wägung 3: Münze 7 vs. 8: " +
141         (result3 == 0 ? "gleich" : (result3 == 1 ? "7 schwerer" : "8
142             schwerer")));
143     if (result3 == 0) { identifiedCoin = 0; identifiedHeavier =
144         false; writer.println("Ergebnis: Münze 1 ist leichter"); }
145     else if (result3 == 1) { identifiedCoin = 6; identifiedHeavier
146         = true; writer.println("Ergebnis: Münze 7 ist schwerer"); }
147     else { identifiedCoin = 7; identifiedHeavier = true;
148         writer.println("Ergebnis: Münze 8 ist schwerer"); }
149 }
150
151 // Vergleich mit der tatsächlichen gefälschten Münze
152 boolean isHeavier = coins[fakeCoin] == 12;
153 return identifiedCoin == fakeCoin && identifiedHeavier == isHeavier;
}

```

A.9 Entscheidungsbaum zur Lösungsstrategie

Abbildung 1 zeigt schematisch den Entscheidungsbaum der festen Strategie. Von der ersten Wägung aus verzweigt sich die Logik abhängig vom Ergebnis in drei Hauptfälle.

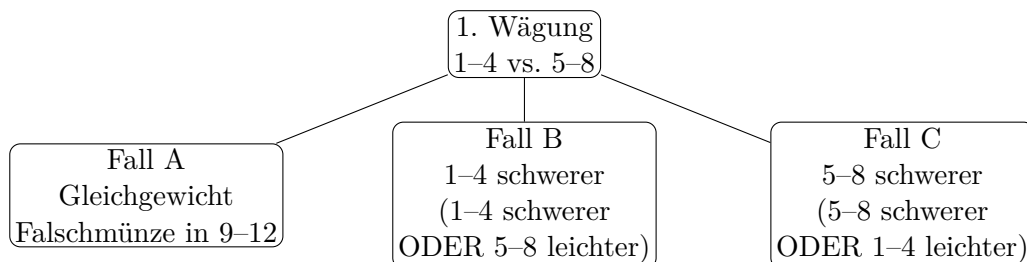


Abbildung 1: Schematischer Entscheidungsbaum ab der ersten Wägung.

Die drei Teilbäume in Abbildung 2, 3 und 4 zeigen, wie die Strategie für jede der drei möglichen Ausgänge der ersten Wägung (Fall A: gleich, Fall B: 1-4 schwerer, Fall C: 5-8 schwerer) mit zwei weiteren Wägungen alle verbleibenden Kandidaten unterscheidet.

A.10 Entscheidungsbaum für Fall A (Falschmünze in 9-12)

Abbildung 2 zeigt den Teil des Entscheidungsbaums, der sich aus Fall A ergibt: Die erste Wägung ergibt Gleichgewicht, also sind Münzen 1-8 echt, und die Falschmünze muss unter Münzen 9-12 sein.

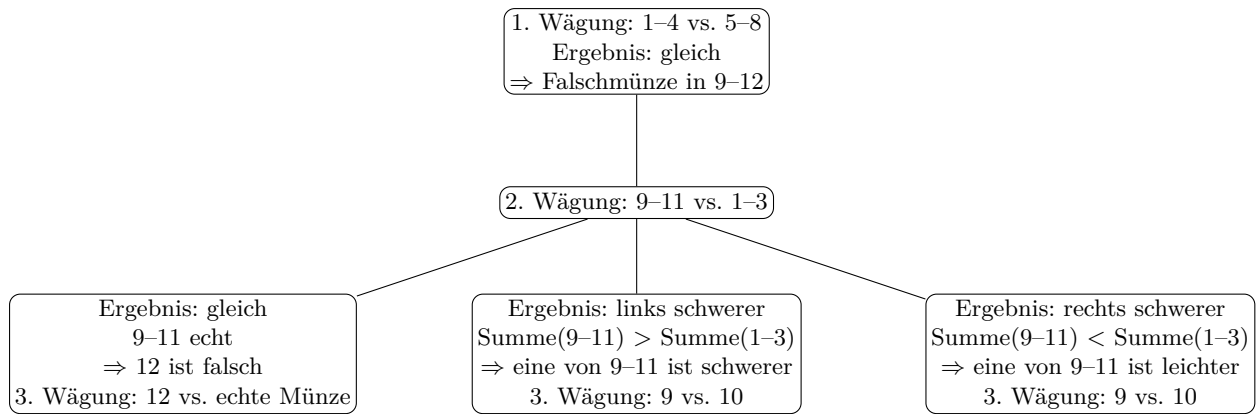


Abbildung 2: Entscheidungsbaum für Fall A: erste Wägung im Gleichgewicht, Falschmünze in 9-12.

A.11 Entscheidungsbaum für Fall B (1-4 schwerer als 5-8)

Abbildung 3 zeigt schematisch den Teil des Entscheidungsbaums, der sich aus Fall B ergibt: Die erste Wägung liefert das Ergebnis, dass die Münzen 1-4 schwerer als die Münzen 5-8 sind.

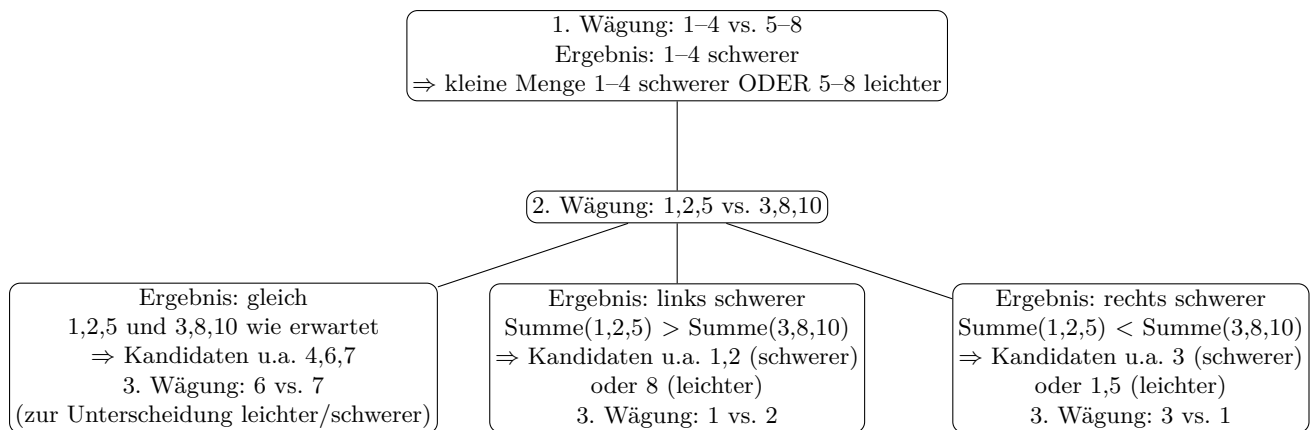


Abbildung 3: Entscheidungsbaum für Fall B: erste Wägung zeigt, dass 1-4 schwerer als 5-8 sind.

A.12 Entscheidungsbaum für Fall C (5-8 schwerer als 1-4)

Abbildung 4 zeigt schematisch den Teil des Entscheidungsbaums, der sich aus Fall C ergibt: Die erste Wägung liefert das Ergebnis, dass die Münzen 5-8 schwerer als die Münzen 1-4 sind.

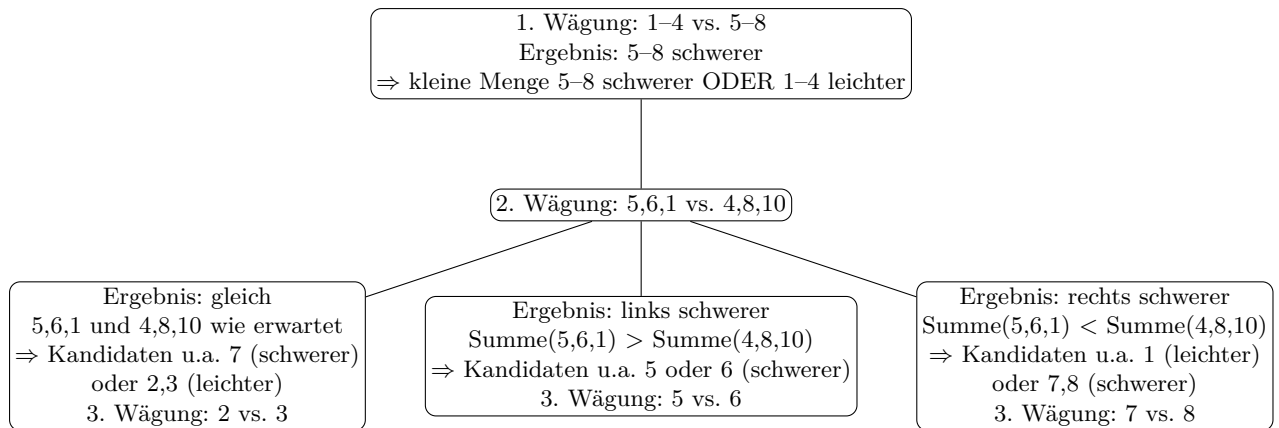


Abbildung 4: Entscheidungsbaum für Fall C: erste Wägung zeigt, dass 5–8 schwerer als 1–4 sind.

Dieser Entscheidungsbaum war bei der Entwicklung der Strategie entscheidend: Erst durch die visuelle Aufteilung in Fälle (A, B, C) und Unterfälle (z.B. A1, A2, A3) wurde klar, wie man alle 24 Konstellationen mit nur drei Wägungen systematisch unterscheiden kann.